Projecto de Bases de Dados (CC2005) - parte 2

1. Elementos do grupo

Grupo nº 46

Nº mecanográfico	**Nome**
202206252	Inês Neves
202205826	Beatriz Fonseca
202208293	Maria Beatriz Moreira

2. Ajustes ao modelo da BD

Tendo em consideração as correções do professor, realizamos os seguintes ajustes ao modelo da Base de Dados:

- ➔ Enriquecimento do modelo, através da adição de dados de outro dataset;
- ➔ Explicação inicial da base de dados mais detalhada;
- ➔ Modificação do diagrama de classes do modelo UML:
    - adição de novas classes e atributos;
    - alteração de algumas associações binárias entre classes;
    - eliminação da classe Open_on e da classe de associação Movement;
- ➔ Reajuste do modelo relacional, de acordo com o novo modelo UML;
- ➔ Explicação do modelo relacional e chaves utilizadas mais detalhada.

3. Povoamento de tabelas

Os datasets utilizados para a base de dados definida, estão disponíveis através dos seguintes **links**:

- https://www.kaggle.com/datasets/mexwell/famous-paintings
  (dataset Famous Paintings)

- https://artlas.huma-num.fr/en/artlas-bases-de-donnees-en-acces-public/
  (dataset Exhibitions)

NOTA: O website Artl@s concede acesso a uma base de dados digital de catálogos de exposições. O conteúdo da base de dados é compilado por dezenas de jovens acadêmicos de todo o mundo, por isso as entradas da base de dados encontram-se em vários idiomas. Dos dados extraídos deste link, restringimos a pesquisa aos anos de 1950-2017, obtendo o dataset que pode ser acedido utilizando o link abaixo:

- https://artlas.huma-num.fr/map/#/result?filters=%5B%5B"city_lower","sw",""%5D,%5B"space_lower","cs",""%5D%5D&type=exhibition&page=1&begin=1950&end=2017
  (dataset Exhibitions)

Alterações realizadas aos datasets:

- Junção dos datasets “Famous Paintings” e “Exhibitions”, formando a base de dados final “Pinturas Famosas”;

- Eliminação das colunas:

  - first_name, middle_names, last_name e style da classe Artist (do Dataset “Famous Paintings”);
  - state, phone, postal e URL da classe Museum (do Dataset “Famous Paintings”);
  - exihibition_group_title (do Dataset “Exhibitions”).

- Não utilizamos as tabelas: canvas_size, image_link, museum_hours, product_size e subject do dataset “Famous Paintings”.

- Tradução dos atributos das tabelas originais para português;
- Renomeação das tabelas (para os nomes que demos posteriormente às respectivas classes);
- Eliminação de algumas entradas da tabela “exposicoes”;
- Adaptação da tabela “exposicoes”, introduzindo dados de artistas em falta na coluna(atributo) “artistasExpostos_id”.

Depois destas alterações, utilizamos a base de dados “Recintos” fornecida em aula. Fizemos “DROP TABLE” para cada uma das tabelas existentes. Renomeámos a base de dados para “PinturasFamosas”, fizemos “CREATE TABLE nome_da_tabela” para cada uma das nossas tabelas.csv e povoámo-las importando as nossas tabelas.csv para a sua table correspondente em “PinturasFamosas”.

Cada tabela ficou com o seguinte número de entradas:

Nome da tabela	**Nº de entradas**
Artistas	421
Pinturas	14776
Museus	57
Paises	35
Exposicoes	660
Artistas_Exposicoes	873
Paises_Exposicoes	662

4. Aplicação Python

“Endpoint”	**Funcionalidade**
/	Página de entrada. Apresenta o número de linhas em Exposicoes, em Artistas, em Pinturas, em Museus e em Países. Permite escolher listar todas as entradas das tabelas anteriormente referidas, ou pesquisar por id ou ainda por expressão em cada uma delas.
/exposicoes/	Página com a listagem das diferentes exposições, apresenta o nome de cada exposição e o exposicao_id que tem associado.
/exposicoes/<string : id>/	Página que mostra a informação disponível de determinado id de pesquisa. Apresenta o exposicaoid, o nome, o ano de início, o ano de fim, o id do pais em que ocorre, o nome da cidade e o local em que ocorre e ainda uma listagem de o/s artista/s que expuseram as suas obras, caso existam.
/exposicoes/search/<expr>/	Página que mostra uma listagem de exposições, dado uma pesquisa por expressão.
/pinturas/	Página com a listagem das diferentes pinturas, apresenta o nome de cada pintura e o pintura_id que tem associado.

/pinturas/<int: id>/	Página que mostra a informação disponível de determinado id de pesquisa. Apresenta o pintura_id, o nome, estilo, museu (caso esteja exposto) e o nome do artista que a pintou.
/pinturas/search/<expr>/	Página que mostra uma listagem de pinturas dado uma pesquisa por expressão.
/artistas/	Página com a listagem dos artistas, apresenta o nome de cada artista e o artista_id que tem associado.
/artistas/<int: id>/	Página que mostra a informação disponível de determinado id de pesquisa. Apresenta o artista_id, o nome, ano de nascimento, ano de óbito e uma listagem de pinturas por cada artista.
/pinturas/search/<expr>/	Página que mostra uma listagem de artistas dado uma pesquisa por expressão.
/museus/	Página com a listagem dos museus, apresenta o nome de cada museu e o museu_id que tem associado.
/museus/<int: id>/	Página que mostra a informação disponível de determinado id de pesquisa. Apresenta o museu_id, o nome, a morada, a cidade e o país em que se localiza o museu.
/museus/search/<expr>/	Página que mostra uma listagem de museus dado uma pesquisa por expressão
/paises/	Página com a listagem dos países, apresenta o nome de cada país e o pais_id que tem associado
/paises/<string : id>/	Página que mostra a informação disponível de determinado id de pesquisa. Apresenta o pais_id, o nome do país, caso tenha, listas os museus desse país e lista as exposições desse país.
/paises/search/<expr>/	Página que mostra uma listagem de países, dado uma pesquisa por expressão.

@APP.route('/')

```
def index():
        stats = db.execute('''
                SELECT * FROM
                (SELECT COUNT(*) n_exposicoes FROM Exposicoes)
                JOIN
                (SELECT COUNT(*) n_artistas FROM Artistas)
                JOIN
                (SELECT COUNT(*) n_pinturas FROM Pinturas)
                JOIN
                (SELECT COUNT(*) n_museus FROM Museus)
                JOIN
                (SELECT COUNT(*) n_paises FROM Paises)
        ''').fetchone()
        logging.info(stats)
        return render_template('index.html',stats=stats)
```

- Através do SELECT e da função COUNT(*) obtém-se a contagem do número de linha de cada uma das tabelas: Exposicoes, Artistas, Pinturas, Museus e Paises. Desta forma, obtém-se o número total de Artistas, de Pinturas, de Museus e de Países. Estas contagens são depois agrupadas com a utilização do JOIN.

@APP.route('/exposicoes/')

```
def list_exposicoes():
        exposicoes= db.execute(
                '''
                SELECT *
                FROM Exposicoes
                ORDER BY nome
                ''').fetchall()
          return render_template('exposicoes-list.html', exposicoes=exposicoes)
```

- Aplicou-se um SELECT * para obter todos os atributos pertencentes à classe Exposicoes, de forma a ter os dados disponíveis para a renderização do html. Os dados foram ordenados por nome de exposição a partir do ORDER BY.

```
@APP.route('/exposicoes/<string:id>/')

def view_by_exposicoes(id):
	exposicao = db.execute(
		'''
		SELECT *
		FROM exposicoes
		WHERE exposicao_id= ?
		''', [id]).fetchone()

	if exposicao is None:
		abort(404, 'Exposicao id {} não existe.'.format(id))

	artistas = db.execute(
		'''
		SELECT a.artista_id, a.nome
		FROM artistas a join Artistas_Exposicoes ae on (a.artista_id = ae.artistas_id)
		JOIN Exposicoes e on (ae.exposicao_id = e.exposicao_id)
		WHERE e.exposicao_id= ?
		ORDER BY a.artista_id
		''', [id]).fetchall()

	paises = db.execute(
		'''
		SELECT p.pais_id, p.nome
		FROM paises p JOIN Paises_Exposicoes pe ON (p.pais_id = pe.pais_id)
		JOIN Exposicoes e ON (pe.exposicao_id = e.exposicao_id)
		WHERE e.exposicao_id = ?
		ORDER BY p.nome
		''', [id]).fetchall()
	 return render_template('exposicao.html', exposicao=exposicao, artistas=artistas,
paises=paises)
```

- Aplicou-se um SELECT *, a partir da tabela Exposicoes, onde o ID da exposicao é igual ao ID do *endpoint*, para se obter todos os dados de Exposicoes correspondentes ao ID.

  Seguidamente, fez-se um SELECT ao ID de artista e ao nome, pela junção da tabela Artistas com Artistas_Exposicoes, com a junção ainda da Tabela Exposicoes, onde o ID seja igual ao do Endpoint. Os resultados foram ordenados pelo ID do artista.

  Por fim, aplicou-se outro SELECT com o nome e o ID do país, sobre a junção da tabela Paises com a tabela Paises_Exposicoes, juntando-se a seguir a tabela Exposicoes, onde o ID seja igual ao ID do Endpoint. Os resultados ordenaram-se pelo nome do país. Assim, foi obtida a informação necessária para gerar o html.

@APP.route('/exposicoes/search/<expr>/')

```
def search_exposicoes(expr):
        search = { 'expr': expr }

        exposicao = db.execute(
                ' SELECT exposicao_id, nome'
                ' FROM exposicoes'
                ' WHERE nome LIKE \'%' + expr + '%\'
                ).fetchall()

         return render_template('exposicao-search.html', search=search, exposicao=exposicao)
```

- Aplicou-se um SELECT ao ID de Exposicoes e ao nome, a partir da tabela Exposicoes onde o nome da exposição contém a expressão “expr” igual a expressão do Endpoint. Assim, obtemos uma listagem das exposições que contém a expressão, caso existam exposições que contenham a expressão do Endpoint.

@APP.route('/pinturas/')

```
def list_pinturas():
        pinturas = db.execute('''
                SELECT pintura_id, nome, artista_id, estilo, museu_id
                FROM Pinturas
                ORDER BY nome
                ''').fetchall()
         return render_template('pinturas-list.html', pinturas=pinturas)
```

- Aplicou-se um SELECT do ID da pintura, do nome da pintura, do ID do artista, do estilo e do ID do museu, sobre a tabela Pinturas. Obteve-se todas as entradas ordenadas pelo nome da pintura.

```
@APP.route('/pinturas/<int:id>/')

def view_by_pinturas(id):
	pinturas = db.execute(
		'''
		SELECT pintura_id, nome, artista_id, estilo, museu_id
		FROM pinturas
		WHERE pintura_id = ?
		''', [id]).fetchone()

	if pinturas is None:
		abort(404, 'Pintura id {} não existe.'.format(id))

	museus = db.execute(
		'''
		SELECT m.museu_id, m.nome
		FROM pinturas p JOIN museus m ON (p.museu_id = m.museu_id)
		WHERE p.pintura_id = ?
		''', [id]).fetchall()

	artistas = db.execute(
		'''
		SELECT a.artista_id, a.nome
		FROM artistas a JOIN pinturas p ON (a.artista_id = p.artista_id )
		WHERE p.pintura_id = ?
		ORDER BY p.nome
		''', [id]).fetchall()
	 return render_template('pintura.html', pinturas=pinturas, museus=museus,
artistas=artistas)
```

- Aplicou-se um SELECT ID da pintura, nome da pintura, ID do artista, estilo da pintura e ID do museu, a partir da tabela Pinturas, onde o ID da pintura é igual ao ID do *endpoint*, para se obter todos os dados de Pinturas correspondentes ao ID.

  Posteriormente, aplicou-se um SELECT ao nome do museu e ao seu ID, a partir da junção da tabela Pinturas com Museus, onde o ID da pintura é igual ao do Endpoint.

  Por fim, fez-se outro SELECT com o ID e o nome do artista, pela junção das tabelas Artistas e Pinturas, onde o ID da pintura fosse igual ao ID do Endpoint. Ordenaram-se os resultados obtidos pelo nome da pintura. Assim, foi obtida a informação necessária para gerar o html.

@APP.route('/pinturas/search/<expr>/')

```
def search_pinturas(expr):
        search = { 'expr': expr }

        pinturas = db.execute(
                ' SELECT pintura_id, nome'
                ' FROM pinturas'
                ' WHERE nome LIKE \'%' + expr + '%\'
                ).fetchall()

            return render_template('pintura-search.html', search=search, pinturas=pinturas)
```

- Aplicou-se um SELECT ao ID de Pinturas e ao nome, a partir da tabela Pinturas onde o nome da pintura contém a expressão “expr” igual a expressão do Endpoint. Assim, obtivemos uma listagem das pinturas que contém a expressão do Endpoint, caso estas existam.

@APP.route ('/artistas/')

```
def list_artistas():
        artistas= db.execute(
                '''
                SELECT *
                FROM Artistas
                ORDER BY nome
                ''').fetchall()
            return render_template ('artistas-list.html',artistas=artistas)
```

- Aplicou-se um SELECT * para obter todos os atributos pertencentes à tabela Artistas, de forma a ter os dados disponíveis para a renderização do html. Os dados foram ordenados pelo nome do artista a partir do ORDER BY. Assim, obtivemos todas as entradas da tabela Artistas ordenadas pelo nome dos Artistas.

@APP.route('/artistas/<int:id>/')

```
def view_by_artistas(id):
        artista = db.execute(
                '''
                SELECT *
                FROM artistas
                WHERE artista_id= ?
                ''', [id]).fetchone()
```

```
    if artista is None:
        abort(404, 'Artista id {} não existe.'.format(id))

    pintura = db.execute(
        '''
        SELECT p.pintura_id, p.nome , p.estilo
        FROM pinturas p JOIN artistas a ON (a.artista_id = p.artista_id )
        WHERE a.artista_id = ?
        ORDER BY p.nome
        ''', [id]).fetchall()
    return render_template('artista.html', artista=artista, pintura=pintura)
```

- Aplicou-se um SELECT *, a partir da tabela Artistas, onde o ID do artista é igual ao ID do *endpoint*, para se obter todos os dados de Artista correspondentes ao ID.

  Consequentemente, aplicou-se um SELECT ao ID da pintura, ao nome da pintura e ao estilo da pintura, a partir da junção da tabela Pinturas com Artistas, onde o ID do Artista seja igual ao do Endpoint. Os resultados foram obtidos por ordem do nome da pintura. Assim, foi obtida a informação necessária para gerar o html.

@APP.route('/artistas/search/<expr>/')

```
def search_artista(expr):
    search = { 'expr': expr }
    artista = db.execute(
        ' SELECT artista_id, nome'
        ' FROM artistas'
        ' WHERE nome LIKE \'%' + expr + '%\'
        ).fetchall()

    return render_template('artista-search.html', search=search, artista=artista)
```

- Aplicou-se um SELECT ao ID de Artistas e ao nome, a partir da tabela Artistas onde o nome do artista contém a expressão “expr” igual à expressão do Endpoint. Assim, obtivemos uma listagem dos artistas cujos nomes contêm a expressão, caso existam nomes de artistas que contenham a expressão do Endpoint.

@APP.route ('/museus/')

```
def list_museus():
        museus= db.execute(
                '''
                SELECT museu_id, nome
                FROM Museus
                ORDER BY nome
                ''').fetchall()
            return render_template ('museus-list.html', museus=museus)
```

- Aplicou-se um SELECT ao ID do museu e do nome do museu, sobre a tabela Museus. Obteve-se todas as entradas ordenadas pelo nome do museu.

@APP.route('/museus/<int:id>/')

```
def view_by_museus(id):
        museus = db.execute(
                '''
                SELECT *
                FROM Museus m
                WHERE m.museu_id= ?
                ''', [id]).fetchone()
        if museus is None:
                abort(404, 'Museu id {} não existe.'.format(id))

        paises = db.execute(
                '''
                SELECT DISTINCT p.pais_id, p.nome
                FROM paises p JOIN Paises_Exposicoes pe ON (pe.pais_id = p.pais_id)
                JOIN Exposicoes e on (pe.exposicao_id = e.exposicao_id )
                JOIN Museus m on (m.pais_id = p.pais_id)
                WHERE m.museu_id = ?
                ORDER BY p.nome
                ''', [id]).fetchall()

            return render_template ('museus.html', museus=museus, paises=paises)
```

- Aplicou-se um SELECT *, a partir da tabela Museus, onde o ID do museu é igual ao ID do *endpoint*, para se obter todos os dados de Museu correspondentes ao ID.

  De seguida, um SELECT DISTINCT ao ID e ao nome do país, a partir da junção da tabela Paises com Paises_Exposicoes. Foi também feita a junção da tabela Paises_Exposicoes com Exposicoes e, por último, a junção das tabelas Exposicoes e Museus, onde o ID de Museu seja igual ao do Endpoint.

  Ordenou-se o resultado obtido por ORDER BY nome do país. Assim, foi obtida a informação necessária para gerar o html.

@APP.route('/museus/search/<expr>/')

```
def search_museus(expr):
        search = { 'expr': expr }
        museus = db.execute(
                ' SELECT museu_id, nome'
                ' FROM museus'
                ' WHERE nome LIKE \'%' + expr + '%\'
                ).fetchall()
            return render_template('museu-search.html', search=search,museus=museus)
```

- Aplicou-se um SELECT ao ID de Museus e ao nome, a partir da tabela Museus onde o nome do museu contém a expressão “expr” igual a expressão do Endpoint. Assim, obtemos uma listagem dos museus cujos nomes contêm a expressão, caso existam museus que contenham a expressão do Endpoint.

@APP.route ('/paises/')

```
def list_paises():
        paises= db.execute(
                '''
                SELECT *
                FROM paises
                ORDER BY nome
                ''').fetchall()
            return render_template ('paises-list.html', paises=paises)
```

- Aplicou-se um SELECT *, sobre a tabela Paises. Obteve-se todas as entradas ordenadas pelo nome do país.

```
@APP.route('/paises/<string:id>/')

def view_by_paises(id):
	paises = db.execute(
		'''
		SELECT *
		FROM Paises p
		WHERE p.pais_id= ?
		''', [id]).fetchone()

	if paises is None:
		abort(404, 'País id {} não existe.'.format(id))

	museus = db.execute(
		'''
		SELECT m.museu_id, m.nome
		FROM museus m
		WHERE pais_id= ?
		ORDER BY m.museu_id
		''', [id]).fetchall()

	exposicoes = db.execute(
		'''
		SELECT e.exposicao_id, e.nome
		FROM exposicoes e JOIN Paises_Exposicoes pe ON (e.exposicao_id=
pe.exposicao_id)
		JOIN Paises p ON (pe.pais_id = p.pais_id)
		WHERE p.pais_id= ?
		ORDER BY e.exposicao_id
		''', [id]).fetchall()
	 return render_template ('paises.html', paises=paises, museus=museus,
exposicoes=exposicoes)
```

- Aplicou-se um SELECT *, a partir da tabela Paises, onde o ID é igual ao ID do *endpoint*, para se obter todos os dados de Paises correspondentes ao ID.

  Depois, aplicou-se um SELECT ao ID de museu e ao nome, a partir da tabela Museus, onde o ID de país seja igual ao do Endpoint, ordenando os resultados obtidos por ORDER BY o ID do museu.

  Por fim, efetuou-se outro SELECT com o ID e o nome da exposição, pela junção da tabela Exposicoes com a tabela Paises_Exposicoes. De seguida, fez-se a junção das tabelas Paises_Exposicoes e Paises, de forma a que o ID de países seja igual ao ID do Endpoint. Ordenaram-se os resultados por ordem do ID da exposição. Assim, foi obtida a informação necessária para gerar o html.

```
@APP.route('/paises/search/<expr>/')

def search_paises(expr):
        search = { 'expr': expr }

        paises = db.execute(
                ' SELECT pais_id, nome'
                ' FROM paises'
                ' WHERE nome LIKE \'%' + expr + '%\"
                ).fetchall()

          return render_template('paises-search.html', search=search,paises=paises)
```

- Aplicou-se um SELECT ao ID de Paises e ao nome, a partir da tabela Paises onde o nome do país contém a expressão “expr” igual a expressão do Endpoint. Assim, obtemos uma listagem dos paises cujos nomes contêm a expressão, caso existam paises que contenham a expressão do Endpoint.